PROJECT REPORT ON

PARKING SLOT ALLOCATION AND VEHICLE ENTRY TRACKING SYSTEM

Submitted in partial fulfillment of the requirements for the course

JAVA PROGRAMMING

Submitted by

MOHAMED HILAL

REGISTER NUMBER: 25BCS235

Under the Guidance of

MR. DHARANEESH

Technical Trainer
Technical Department
KG College of Arts and Science

Academic Year: 2025–2026
Semester: II
October 2026

DECLARATION

I hereby declare that the project entitled **“Parking Slot Allocation and Vehicle Entry Tracking System”**, submitted in partial fulfillment of the requirements for the course Java Programming, is a project report prepared by me under the guidance of **Mr. Dharaneesh**, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this report has been prepared for academic purposes and that any external sources used have been acknowledged wherever applicable. The implementation details and sample outputs in this report describe a proposed console-based Java prototype and should be updated to match the final program that is developed and tested.

Date: _____

Place: KG College of Arts and Science

Signature of the Student: _____

CERTIFICATE

This is to certify that the project entitled **“Parking Slot Allocation and Vehicle Entry Tracking System”** is submitted by **Mohamed Hilal** (Register Number: **25BCS235**), B.Sc. Computer Science, KG College of Arts and Science, in partial fulfillment of the requirements for the course Java Programming. The project report is prepared under the guidance of **Mr. Dharaneesh**, Technical Trainer, Technical Department, KG College of Arts and Science.

The date and approval details may be completed by the department at the time of submission.

Place: KG College of Arts and Science

Date: _____

Project Guide	Head of the Department
Mr. Dharaneesh	_____
Technical Trainer	Authorized Signatory

TABLE OF CONTENTS

S. No.	Topic
1	Abstract
2	Introduction
2.1	Objective
2.2	Overview
2.3	Features
3	Program Analysis
3.1	Problem Definition
3.2	Project Overview
3.3	Scope and Limitations
4	System Design
4.1	Modules
5	Methodology
5.1	Problem Analysis
5.2	System Design
5.3	Program Development
5.4	Testing and Validation
5.5	Result and Documentation
5.6	Algorithm
5.7	Flowchart
6	Source Code
7	Sample Input and Output
8	Discussion and Conclusion
8.1	Discussion
8.2	Conclusion
9	References

ABSTRACT

The Parking Slot Allocation and Vehicle Entry Tracking System is a Java-based application proposed to organize parking-space assignment and maintain vehicle entry and exit records. In conventional parking areas, drivers and attendants may have difficulty identifying vacant spaces, tracking which vehicle occupies each slot, and maintaining reliable time records. These issues can cause delays, congestion, and errors in manual registers. The proposed system maintains a list of parking slots and their current status, accepts vehicle details at entry, allocates an available slot, and records the entry time. When the vehicle leaves, the system records the exit time and changes the slot status back to available. The project can be implemented as a menu-driven console application using Core Java concepts such as classes, arrays or collections, loops, conditional statements, methods, and date-time handling. The prototype described in this report focuses on basic allocation and tracking; database integration, automated number-plate recognition, sensors, and online reservations are possible future improvements. The main objective is to demonstrate how Java programming can support more systematic and accurate parking management.

Keywords: Parking management, slot allocation, vehicle entry, exit tracking, Java, occupancy status.

1. INTRODUCTION

Parking management is an important activity in colleges, offices, commercial buildings, hospitals, and public facilities. When the number of vehicles increases, manually checking vacant spaces and writing down entry and exit details becomes difficult. Paper-based registers can be incomplete, and attendants may accidentally assign an occupied slot or fail to update a slot after a vehicle leaves.

The Parking Slot Allocation and Vehicle Entry Tracking System aims to provide a simple computerized approach. The application stores a set of parking slots, identifies slots marked as available, assigns one to an arriving vehicle, and records the time of entry. At departure, the active parking session is closed, the exit time is saved, and the slot is made available for another vehicle. A menu-driven interface allows an operator to perform these operations in a consistent sequence.

1.1 Objective

• To develop a Java-based prototype for parking-slot allocation.

• To maintain the current status of each parking slot.

• To record vehicle registration number, allocated slot, entry time, and exit time.

• To prevent allocation of a slot that is already occupied.

• To display parking availability and active parking sessions.

• To demonstrate the use of programming logic in a practical management problem.

1.2 Overview

The system is designed as a menu-driven application. The operator can view slot availability, register an arriving vehicle, allocate an available slot, look up an active vehicle, and record a vehicle exit. Each active parking session links a vehicle to one slot. For a basic academic prototype, data may be held in memory while the program is running; persistent storage can be added later.

1.3 Features

• Display all parking slots and their status.

• Register a vehicle using its registration number.

• Allocate the first available slot or another suitable vacant slot.

• Record entry and exit date-time values.

• Release a slot when a vehicle exits.

• Search active parking information by vehicle number.

• Show occupied, available, and total slot counts.

2. PROGRAM ANALYSIS

Program analysis identifies the operational problem, the data required, and the expected behavior before coding begins. For this project, the system must keep slot status consistent with vehicle sessions and must provide clear messages when parking is full, a vehicle is not found, or an input is invalid.

2.1 Problem Definition

Common problems in a manual parking process include:

• Drivers and attendants may not know which spaces are vacant.

• A vehicle may be directed to a slot that is already occupied.

• Entry and exit times may be missing or recorded incorrectly.

• Searching paper records for a particular vehicle can be slow.

• Slot availability may not be updated promptly when a vehicle leaves.

The proposed system addresses these problems by maintaining slot status and parking-session records through a consistent program workflow.

2.2 Project Overview

The application maintains parking-slot records and active vehicle sessions. When a vehicle arrives, the operator enters its registration number. The program checks for an existing active session, finds an available slot, marks the slot occupied, and saves the entry time. On exit, the operator searches for the active session, records the exit time, closes the session, and marks the slot available. A status view summarizes current occupancy.

2.3 Scope and Limitations

Scope

• Suitable as a learning prototype for college, office, or small parking facilities.

• Supports basic allocation, vehicle entry/exit tracking, and occupancy display.

• Can be extended with a graphical interface, database, user login, or fee calculation.

Limitations of the basic prototype

• If records are stored only in memory, they are lost when the program closes.

• Vehicle number and entry/exit times are entered by the operator rather than detected automatically.

• No live sensors, camera-based number-plate recognition, online reservation, or payment integration is included by default.

• A single local program does not provide multi-gate synchronization unless additional architecture is implemented.

3. SYSTEM DESIGN

System design defines the program's structure, data, and interactions. The system can be separated into small modules so that each function is easier to implement, test, and maintain. The following design describes a console-based Java prototype; exact class names can be adapted during implementation.

3.1 Modules

1. Slot Management Module

Creates or displays slot IDs and stores each slot's status, such as AVAILABLE or OCCUPIED.

2. Vehicle Registration Module

Accepts a vehicle registration number and checks that the input is present and valid enough for the prototype.

3. Slot Allocation Module

Searches for an available slot, assigns it to the arriving vehicle, and changes its status to OCCUPIED.

4. Entry Tracking Module

Creates a parking session and records the vehicle number, slot ID, and entry date-time.

5. Exit Tracking Module

Finds the active session, records the exit date-time, closes the session, and releases the slot.

6. Availability and Search Module

Displays free and occupied slots and allows the operator to search for a vehicle's active parking details.

7. Exit / Menu Module

Repeats the menu until the operator chooses to end the application.

3.2 Data Design

Entity	Suggested fields	Purpose
ParkingSlot	slotId, status, slotType	Stores each parking space and its current availability.
Vehicle	vehicleNumber, vehicleType (optional)	Identifies the vehicle being parked.
ParkingSession	sessionId, vehicleNumber, slotId, entryTime, exitTime, active	Records one visit from entry until exit.

Important rule: an occupied slot must not be allocated again, and a vehicle should not have two simultaneous active sessions in the same facility.

4. METHODOLOGY

The development process follows the same report structure as the reference document: problem analysis, system design, program development, testing and validation, and result documentation.

4.1 Problem Analysis

• Review how vehicles are assigned to spaces and how entry/exit information is recorded.

• Identify required data: slot ID, vehicle number, entry time, exit time, and slot status.

• Define expected responses for normal and exceptional cases, including a full parking area.

4.2 System Design

• Design a menu for viewing slots, vehicle entry, vehicle exit, search, and program termination.

• Choose data structures to represent parking slots and parking sessions.

• Define the rule that a slot is released only after a valid exit is recorded.

4.3 Program Development

Suggested technologies

• Java (JDK 8 or later)

• VS Code, Eclipse, or IntelliJ IDEA

• Java standard library for collections and date-time handling

Java concepts

• Classes and objects

• Arrays or ArrayList collections

• Loops and conditional statements

• Methods for separate modules

• LocalDateTime for entry and exit timestamps

Implementation should proceed module by module. First create the slot list, then implement availability checking, vehicle entry, exit processing, search, and summary display. Add input validation and test each operation as it is completed.

4.4 Testing and Validation

Test case	Expected result
A free slot exists	Vehicle is assigned a free slot and its status becomes OCCUPIED.
All slots occupied	System displays a parking-full message and does not create an allocation.
Vehicle exits	Exit time is recorded and the associated slot becomes AVAILABLE.
Unknown vehicle exits	System reports that no active session was found.
Duplicate active vehicle entry	System prevents a second active session for the same vehicle.
Invalid menu option	System displays an error and allows the operator to try again.

4.5 Result and Documentation

When implemented and tested, the system should provide a clear record of which slots are available, which are occupied, and which vehicle is associated with each active slot. The report documents the objectives, modules, algorithm, representative Java implementation, and sample console interaction. The final result section should be updated with screenshots and actual test outcomes from the student's running program.

4.6 Algorithm

1. Start the program.

2. Initialize the parking-slot list and mark every slot AVAILABLE.

3. Display the main menu.

4. If the user selects Vehicle Entry:

a. Read and validate the vehicle registration number.

b. Check whether the vehicle already has an active session.

c. Find an AVAILABLE slot.

d. If found, assign it, record entry time, and mark it OCCUPIED; otherwise display Parking Full.

5. If the user selects Vehicle Exit:

a. Read the vehicle registration number and find its active session.

b. If found, record exit time, close the session, and mark the slot AVAILABLE.

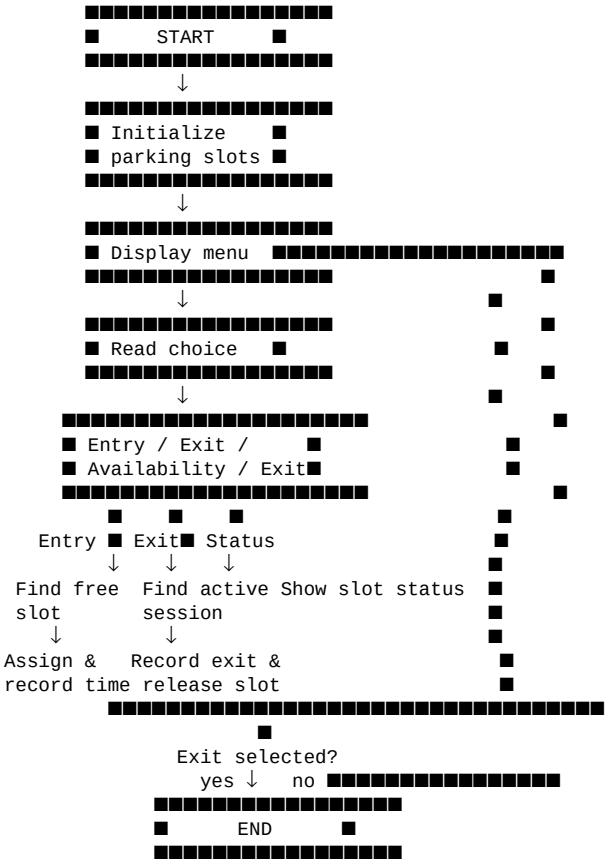
c. Otherwise display Vehicle Not Found.

6. If the user selects Availability, display slot status and occupancy totals.

7. Repeat the menu until Exit is selected.

8. End the program.

4.7 Flowchart



5. SOURCE CODE

The following is a representative console-based Java implementation for the documented workflow. It uses an in-memory list, so data is reset when the program restarts. Compile and test it in your Java environment, and adjust it if your final project has additional requirements.

```
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class ParkingSystem {
    static final Scanner sc = new Scanner(System.in);
    static final DateTimeFormatter FORMAT =
        DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm:ss");

    static class ParkingSlot {
        int id;
        boolean occupied;

        ParkingSlot(int id) {
            this.id = id;
            this.occupied = false;
        }
    }

    static class ParkingSession {
        String vehicleNumber;
        int slotId;
        LocalDateTime entryTime;
        LocalDateTime exitTime;
        boolean active;

        ParkingSession(String vehicleNumber, int slotId) {
            this.vehicleNumber = vehicleNumber;
            this.slotId = slotId;
            this.entryTime = LocalDateTime.now();
            this.active = true;
        }
    }

    static List<ParkingSlot> slots = new ArrayList<>();
    static List<ParkingSession> sessions = new ArrayList<>();

    public static void main(String[] args) {
        for (int i = 1; i <= 10; i++) {
            slots.add(new ParkingSlot(i));
        }

        int choice;
        do {
            System.out.println("\\n=== PARKING SLOT ALLOCATION SYSTEM ===");
            System.out.println("1. Vehicle Entry");
            System.out.println("2. Vehicle Exit");
            System.out.println("3. View Slot Availability");
            System.out.println("4. Search Active Vehicle");
            System.out.println("0. Exit");
            System.out.print("Enter choice: ");
            choice = readInt();

            switch (choice) {
                case 1: vehicleEntry(); break;
                case 2: vehicleExit(); break;
                case 3: displaySlots(); break;
                case 4: searchVehicle(); break;
                case 0: System.out.println("System closed."); break;
                default: System.out.println("Invalid choice. Try again.");
            }
        } while (choice != 0);
        sc.close();
    }

    static int readInt() {
        while (!sc.hasNextInt()) {
            System.out.print("Enter a valid number: ");
            sc.next();
        }
```

```
        int value = sc.nextInt();
        sc.nextLine();
        return value;
    }

    static void vehicleEntry() {
        System.out.print("Enter vehicle number: ");
        String number = sc.nextLine().trim().toUpperCase();

        if (number.isEmpty()) {
            System.out.println("Vehicle number cannot be empty.");
            return;
        }
        for (ParkingSession session : sessions) {
            if (session.active && session.vehicleNumber.equals(number)) {
                System.out.println("This vehicle already has an active session.");
                return;
            }
        }

        for (ParkingSlot slot : slots) {
            if (!slot.occupied) {
                slot.occupied = true;
                ParkingSession session =
                    new ParkingSession(number, slot.id);
                sessions.add(session);
                System.out.println("Entry recorded successfully.");
                System.out.println("Vehicle: " + number);
                System.out.println("Allocated slot: " + slot.id);
                System.out.println("Entry time: " +
                    session.entryTime.format(FORMAT));
                return;
            }
        }
        System.out.println("Parking Full. No slot is available.");
    }

    static void vehicleExit() {
        System.out.print("Enter vehicle number: ");
        String number = sc.nextLine().trim().toUpperCase();

        for (ParkingSession session : sessions) {
            if (session.active && session.vehicleNumber.equals(number)) {
                session.exitTime = LocalDateTime.now();
                session.active = false;
                for (ParkingSlot slot : slots) {
                    if (slot.id == session.slotId) {
                        slot.occupied = false;
                        break;
                    }
                }
                System.out.println("Exit recorded successfully.");
                System.out.println("Slot released: " + session.slotId);
                System.out.println("Entry time: " +
                    session.entryTime.format(FORMAT));
                System.out.println("Exit time: " +
                    session.exitTime.format(FORMAT));
                return;
            }
        }
        System.out.println("No active parking session found.");
    }

    static void displaySlots() {
        int available = 0;
        System.out.println("\\n--- SLOT STATUS ---");
        for (ParkingSlot slot : slots) {
            String status = slot.occupied ? "OCCUPIED" : "AVAILABLE";
            System.out.println("Slot " + slot.id + ": " + status);
            if (!slot.occupied) available++;
        }
        System.out.println("Total slots: " + slots.size());
        System.out.println("Available: " + available);
        System.out.println("Occupied: " + (slots.size() - available));
    }

    static void searchVehicle() {
        System.out.print("Enter vehicle number: ");
        String number = sc.nextLine().trim().toUpperCase();

        for (ParkingSession session : sessions) {
```

```
            if (session.active && session.vehicleNumber.equals(number)) {
                System.out.println("Vehicle: " + session.vehicleNumber);
                System.out.println("Slot: " + session.slotId);
                System.out.println("Entry time: " +
                    session.entryTime.format(FORMAT));
                return;
            }
        }
        System.out.println("No active parking session found.");
    }
}
```

6. SAMPLE INPUT AND OUTPUT

The following transcript illustrates expected behavior for the sample implementation. It is an illustrative example, not a screenshot from an executed program.

```
=== PARKING SLOT ALLOCATION SYSTEM ===
1. Vehicle Entry
2. Vehicle Exit
3. View Slot Availability
4. Search Active Vehicle
0. Exit
Enter choice: 1
Enter vehicle number: TN38AB1234
Entry recorded successfully.
Vehicle: TN38AB1234
Allocated slot: 1
Entry time: 23-10-2026 10:15:00

=== PARKING SLOT ALLOCATION SYSTEM ===
1. Vehicle Entry
2. Vehicle Exit
3. View Slot Availability
4. Search Active Vehicle
0. Exit
Enter choice: 3

--- SLOT STATUS ---
Slot 1: OCCUPIED
Slot 2: AVAILABLE
Slot 3: AVAILABLE
...
Total slots: 10
Available: 9
Occupied: 1

=== PARKING SLOT ALLOCATION SYSTEM ===
Enter choice: 2
Enter vehicle number: TN38AB1234
Exit recorded successfully.
Slot released: 1
Entry time: 23-10-2026 10:15:00
Exit time: 23-10-2026 11:05:00
```

For final submission, replace or supplement this sample with actual screenshots from your own program running in VS Code, Eclipse, or IntelliJ IDEA.

7. DISCUSSION AND CONCLUSION

7.1 Discussion

The Parking Slot Allocation and Vehicle Entry Tracking System provides a structured approach to managing basic parking operations. Slot allocation is based on current availability, and an entry creates an active parking session containing the vehicle number, slot ID, and entry timestamp. The exit operation closes that session, stores the exit timestamp, and releases the slot for reuse. These steps help reduce reliance on handwritten records and make it easier to check the current parking status.

The project demonstrates Java concepts including classes, objects, collections, loops, conditional statements, methods, and date-time handling. Testing should verify that the same slot is not assigned twice, that a full parking area is handled correctly, and that only vehicles with active sessions can release slots. Because the basic prototype uses in-memory storage and operator-entered details, it should be viewed as a foundation rather than a complete production parking platform.

7.2 Conclusion

The project presents a practical Java-based design for allocating parking spaces and tracking vehicle entry and exit. It organizes slot availability, associates each parked vehicle with a slot, and records the times of entry and departure. The menu-driven structure makes the workflow straightforward for a small facility and supports the learning objectives of modular programming and input validation.

Future versions may introduce a database to preserve records, a graphical dashboard, multiple parking zones, vehicle-type matching, access control, sensors, QR scanning, automatic number-plate recognition, and reports. These enhancements would need to be implemented and tested before being claimed as system capabilities.

8. REFERENCES

1. Oracle. Java Platform, Standard Edition Documentation. https://docs.oracle.com/en/java/

2. Oracle. Java Tutorials: Classes and Objects. https://docs.oracle.com/javase/tutorial/java/javaOO/

3. Oracle. Java Date-Time API. https://docs.oracle.com/javase/8/docs/api/java/time/package-summary.html

4. Herbert Schildt, Java: The Complete Reference, McGraw-Hill.

5. Kathy Sierra and Bert Bates, Head First Java, O'Reilly Media.

Submission note: Please confirm the academic year, semester, guide designation, and department's preferred certificate wording before printing. Add actual program screenshots and update test results after running the code.